

Air University



DSA (Lab)

Year 2025

Mr. Ali Raza

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: December 22, 2025

Assignment # 04

Question

Code

```
#include <iostream>
#include <cstring>

using namespace std;

const int MAX_V = 100;
const int MAX_E = 100;

class Graph
{
private:
    int V; // Number of vertices
    int adj[MAX_V][MAX_E]; // Adjacency list using 2D array
    int adjSize[MAX_V]; // Size of each adjacency list
    int order[MAX_V]; // Array to store traversal order
    int orderSize; // Size of the order array

    // Recursive utility function for DFS
    void DFSUtil(int u, bool visited[])
    {
        visited[u] = true;
        order[orderSize++] = u; // Add to order

        for (int i = 0; i < adjSize[u]; i++)
        {
            int v = adj[u][i];
            if (!visited[v])
            {
                DFSUtil(v, visited);
            }
        }
    }

    // Helper function to print the traversal order
    void printOrder(int order[], int size)
    {
        cout << "[";
        for (int i = 0; i < size; ++i)
        {
            if (i > 0)
                cout << ", ";
            cout << order[i];
        }
        cout << "]";
    }
}
```

```
// Helper function to print the list of visited vertices
void printVisited(bool visited[])
{
    cout << "[";
    bool first = true;
    for (int i = 0; i < V; i++)
    {
        if (visited[i])
        {
            if (!first)
                cout << ", ";
            cout << i;
            first = false;
        }
    }
    cout << "]";
}

public:
    Graph(int v) // Constructor
    {
        V = v;
        memset(adjSize, 0, sizeof(adjSize)); // Initialize adjSize to 0
        orderSize = 0;
    }

    void addEdge(int u, int v) // Add an edge to the undirected graph
    {
        adj[u][adjSize[u]++] = v;
        adj[v][adjSize[v]++] = u;
    }

    void BFS(int source) // BFS starting from source vertex
    {
        bool visited[MAX_V] = {false};
        int q[MAX_V]; // Manual queue using array
        int front = 0, rear = -1;
        orderSize = 0; // Reset order size

        visited[source] = true;
        q[++rear] = source;

        while (front <= rear)
        {
            int u = q[front++];
            order[orderSize++] = u; // Add to order

            for (int i = 0; i < adjSize[u]; i++)
            {
                int v = adj[u][i];
                if (!visited[v])
                {
```

```
        visited[v] = true;
        q[++rear] = v;
    }
}
cout << "BFS from " << source << ": ";
printOrder(order, orderSize);

cout << "\nVisited vertices: ";
printVisited(visited);
cout << endl
    << endl;
}

void DFS(int source) // DFS starting from source vertex
{
    bool visited[MAX_V] = {false};
    orderSize = 0; // Reset order size

    DFSUtil(source, visited);
    cout << "DFS from " << source << ": ";
    printOrder(order, orderSize);

    cout << "\nVisited vertices: ";
    printVisited(visited);
    cout << endl
        << endl;
}

};

int main()
{
    Graph g(5); // Create a graph with 5 vertices

    // Add edges
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 3);
    g.addEdge(1, 4);
    g.addEdge(2, 4);

    g.BFS(0);

    g.DFS(0);

    return 0;
}
```

Output

```
BFS from 0: [0, 1, 2, 3, 4]
Visited vertices: [0, 1, 2, 3, 4]

DFS from 0: [0, 1, 3, 4, 2]
Visited vertices: [0, 1, 2, 3, 4]
```

THE END
